
Systolic arrays

Matrix multiplication becomes efficient when data moves through local, carefully timed paths.

Li Shang

ML Systems, Fall 2026

Li Shang

THE ARCHITECTURAL PROBLEM

The bottleneck is feeding the multipliers

Each c_{ij} needs K multiply-accumulate operations. If every processing element fetches both operands and its partial sum from central memory, thousands of multipliers create thousands of simultaneous requests. The hardware must make matrix multiplication's reuse local.

Reuse becomes local movement through the array

A value enters once, moves one hop per cycle, and contributes to several outputs.



Rows of A enter from the left. Columns of B enter from the top.

Each PE multiplies the arriving pair, adds it to a local sum, then passes A right and B down.

Each PE keeps one partial sum. After K matching pairs, that sum is one element of C.

Global memory injects each value once. Neighbor-to-neighbor transfers provide the remaining reuse.

A 3×3 wavefront needs staggered inputs

Cycle 1

PE11	PE12	PE13
PE21	PE22	PE23
PE31	PE32	PE33

Cycle 2

PE11	PE12	PE13
PE21	PE22	PE23
PE31	PE32	PE33

Cycle 3

PE11	PE12	PE13
PE21	PE22	PE23
PE31	PE32	PE33

Why the inputs are staggered

PE₂₂ sits one hop from the left and top edges. Delaying row 2 of A and column 2 of B by one cycle makes a_{21} meet b_{12} there. Farther PEs start later for the same reason.

The figure marks valid operand pairs. It does not show the later drain phase.

Red cells perform a valid multiply-accumulate in that cycle. Gray cells are waiting.

The array fills, reaches steady state, then drains

Startup and tail cost matter for small tiles. Large matrix products spend more time in steady state.

Step 1

Skew inputs

Delay each stream so matching operands meet in the correct PE.

Cycle alignment

Step 2

Fill

The active diagonal expands while many PEs still wait for valid pairs.

About $O(N)$ cycles

Step 3

Steady state

Every PE performs one multiply-accumulate in each cycle.

N^2 MACs per cycle

Step 4

Drain

After the last inputs enter, activity leaves along a diagonal.

About $O(N)$ cycles

Specialization creates predictable limits

- | | | |
|----|--------------------------|---|
| 01 | Shape match | Small matrices and edge tiles leave many processing elements idle. |
| 02 | Pipeline overhead | Fill and drain each cost about $O(N)$ cycles, which can dominate short computations. |
| 03 | Irregular work | Sparsity, gather and scatter, branching, and data-dependent routing disrupt the regular flow. |
| 04 | System balance | Boundary memory bandwidth and non-matrix operations can become the next bottlenecks. |